

Sistema de Sinalização Embarcado

ESP32, MicroPython, LEDs, Botão e OLED
SSD1306

Candidato:

Sergio Cordero Calvimontes

Plataforma: Wokwi (Simulador)

Linguagem: MicroPython

Arquitetura: Assíncrona (asyncio)

Visão Geral & Hardware

Objetivo do Projeto

Desenvolver e simular um sistema embarcado responsivo com **ESP32** integrando tópicos de:

- Instrumentação
- Eletrônica
- Lógica de Programação

Placa-Alvo

Espresif ESP32-DevKitC V4

Módulo recomendado: **ESP32-WROOM-32E**.

(Evitar WROVER: usa internamente os GPIOs 16/17 para a PSRAM.)

Conexões dos GPIOs

- **LED Vermelho:** GPIO 2 (resistor 220 $[\Omega]$)
- **LED Verde:** GPIO 4 (resistor 220 $[\Omega]$)
- **Botão pulsador:** GPIO 17 (PULL_DOWN)
- **OLED SCL:** GPIO 25 (I²C *clock*)
- **OLED SDA:** GPIO 16 (I²C *data*)

Funcionamento

LED Vermelho (GPIO 2)

Pisca continuamente a cada **500 [ms]**, em uma tarefa assíncrona própria.

Resposta ao Botão (GPIO 17)

Botão	LED Verde	OLED
Solto (LOW)	Apagado	"Boa sorte!"
Pressionado	Aceso	"Conseguí"

Duas Tarefas, Nunca Bloqueantes

Exclusivamente **asyncio** do MicroPython, sem *superlaço* (*superloop*) nem `time.sleep()`: as duas tarefas acima rodam como **corrotinas** cooperativas, e nenhuma bloqueia a outra.

OLED: Atualização Dinâmica

Atualizado apenas quando o estado **estável** do botão muda – evita tráfego I²C desnecessário e cintilação (*flickering*).

Validação & Conclusão

Estratégia de Testes Isolados

Testes progressivos em `tests/`, do mais simples ao mais completo: LED vermelho (bloqueante e com `asyncio`), botão + LED verde, OLED I²C (varredura e diagnóstico gráfico) e integração final em `main.py`.

Requisitos Atendidos

- ✓ LED vermelho: ciclo de 500 [ms], contínuo e não bloqueante.
- ✓ LED verde: reflete o estado do botão.
- ✓ OLED: mensagem correta por estado, com atualização dinâmica.
- ✓ Execução concorrente via `asyncio`.

Links para Acesso

Wokwi: <https://wokwi.com/projects/471528241540407297>

GitHub: <https://github.com/Argus-Kepheus/cess-uff>